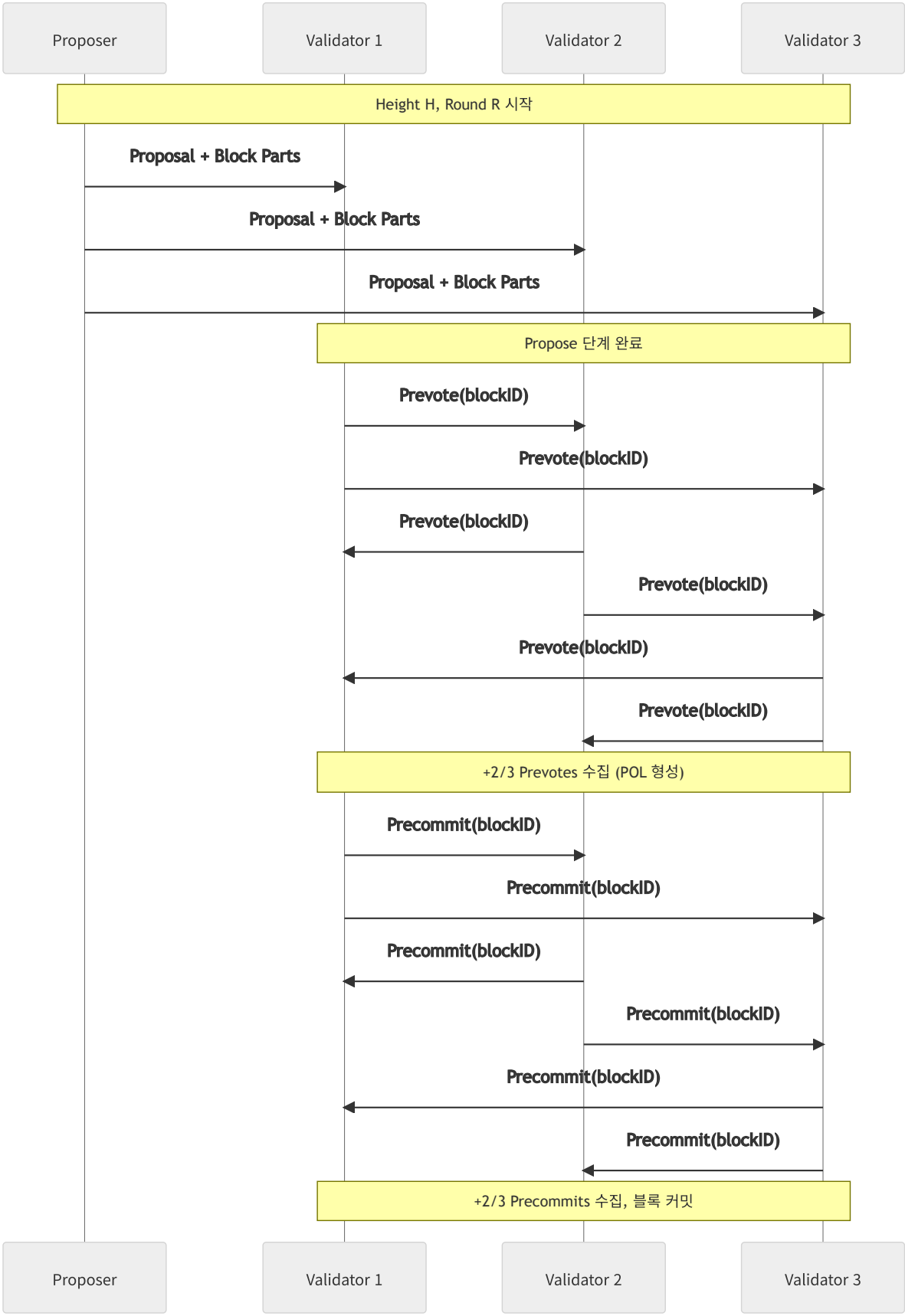
합의 단계별 상세 분석

개요

이 문서는 Tendermint 합의 알고리즘의 각 단계를 상세히 분석합니다. 각 단계에서 일어나는 구체적인 동작, 검증 로직, 메시지 교환 과정을 코드 레벨에서 설명합니다.

합의 단계 흐름도


Proposer
Validator 1
Validator 2
Validator 3
Height H, Round R 시작
Proposal + Block Parts
Proposal + Block Parts
Proposal + Block Parts
Propose 단계 완료
Prevote(blockID)
Prevote(blockID)
Prevote(blockID)
Prevote(blockID)
Prevote(blockID)
Prevote(blockID)
+2/3 Prevotes 수집 (POL 형성)
Precommit(blockID)
Precommit(blockID)
Precommit(blockID)
Precommit(blockID)
Precommit(blockID)
Precommit(blockID)
+2/3 Precommits 수집, 블록 커밋
Proposer
Validator 1
Validator 2
Validator 3

1. 제안 단계 (Propose Step)

1.1 단계 진입 조건

제안 단계는 다음 상황에서 시작됩니다:

- 새로운 라운드가 시작될 때 (`enterNewRound` → `enterPropose`)
- 이전 높이의 커밋이 완료된 후 (`enterNewHeight` → `enterPropose`)

1.2 제안자 선택

제안자는 다음 알고리즘으로 선택됩니다:

```
// 라운드 로빈 방식으로 제안자 선택
proposer := validators.GetProposer()
// 라운드가 증가하면 다음 밸리데이터로 순환
validators.IncrementProposerPriority(1)
```

1.3 블록 제안 과정

1.3.1 블록 생성

```
func (cs *State) createProposalBlock() (*types.Block, error) {
// 1. 메모리풀에서 트랜잭션 수집
```

```
txs := cs.mempool.ReapMaxBytesMaxGas(maxBytes, maxGas)
// 2. 증거(Evidence) 수집
evidence := cs.evpool.PendingEvidence(maxEvidenceBytes)
// 3. 블록 생성
block := cs.blockExec.CreateProposalBlock(
cs.Height, cs.state, commit, proposerAddr, txs, evidence
)
return  block, nil
}
```

1.3.2 제안 메시지 생성 및 전파

```
func (cs *State) enterPropose(height  int64, round  int32) {
// 제안자인지 확인
if !cs.isProposer() {
// 제안자가 아니면 타임아웃 설정 후 대기
cs.scheduleTimeout(cs.config.Propose(round), height, round, RoundStepPropose)
return
}
// 블록 생성
block, blockParts := cs.createProposalBlock()
// 제안 메시지 생성
proposal := &types.Proposal{
Height: height,
```

```
Round: round,
BlockID: types.BlockID{Hash: block.Hash(), PartSetHeader: blockParts.Header()},
POLRound: cs.ValidRound, // Proof of Lock 라운드
Timestamp: time.Now(),
}
// 제안에 서명
cs.privValidator.SignProposal(cs.state.ChainID, proposal)
// 네트워크에 전파
cs.sendInternalMessage(msgInfo{&ProposalMessage{proposal}, "", time.Now()})
// 블록 파트들도 전파
for i := 0; i < blockParts.Total(); i++ {
part := blockParts.GetPart(i)
cs.sendInternalMessage(msgInfo{&BlockPartMessage{height, round, part}, "", time.
}
}
```

1.4 제안 검증

다른 밸리데이터들이 제안을 받으면 다음과 같이 검증합니다:

```
func (cs *State) setProposal(proposal *types.Proposal, receiveTime time.Time) er
// 1. 기본 검증
if proposal.Height != cs.Height || proposal.Round != cs.Round {
return ErrInvalidProposal
```

```
}

// 2. 제안자 검증

expectedProposer := cs.Validators.GetProposer()

if !proposal.ValidateBasic() {

return  ErrInvalidProposal

}

// 3. 서명 검증

if !expectedProposer.PubKey.VerifySignature(proposal.SignBytes(cs.state.ChainID)

return  ErrInvalidSignature

}

// 4. PBTS (Proposer-Based Timestamp) 검증

if  cs.isPBTSEnabled() {

if !cs.proposalIsTimely() {

return  ErrUntimelyProposal

}

}

cs.Proposal = proposal

cs.ProposalReceiveTime = receiveTime

return  nil

}
```

2. 사전 투표 단계 (Prevote Step)

2.1 단계 진입 조건

사전 투표 단계는 다음 상황에서 시작됩니다:

- 제안이 완전히 수신되었을 때
- 제안 타임아웃이 발생했을 때

2.2 사전 투표 로직

```
func (cs *State) defaultDoPrevote(height int64, round int32) {
// 1. 제안이 없는 경우 nil 투표
if cs.Proposal == nil {
cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
return
}
// 2. 블록을 완전히 받지 못한 경우 nil 투표
if cs.ProposalBlock == nil {
cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
return
}
// 3. POL(Proof of Lock) 검사
if cs.Proposal.POLRound == -1 {
// 새로운 제안인 경우
if cs.LockedRound == -1 {
// 잠긴 블록이 없는 경우
if cs.ValidRound != -1 && cs.ProposalBlock.HashesTo(cs.ValidBlock.Hash()) {
```

```
// 이전에 유효했던 블록과 같으면 투표
cs.signAddVote(types.PrevoteType, cs.ProposalBlock.Hash(), cs.ProposalBlockParts
return
}
// PBTS 검증
if  cs.isPBTSEnabled() && !cs.proposalIsTimely() {
cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
return
}
// 애플리케이션 검증
if  err := cs.blockExec.ValidateBlock(cs.state, cs.ProposalBlock);  err != nil {
cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
return
}
// 모든 검증 통과 시 블록에 투표
cs.signAddVote(types.PrevoteType, cs.ProposalBlock.Hash(), cs.ProposalBlockParts
} else {
// 잠긴 블록이 있는 경우
if  cs.ProposalBlock.HashesTo(cs.LockedBlock.Hash()) {
// 제안된 블록이 잠긴 블록과 같으면 투표
cs.signAddVote(types.PrevoteType, cs.ProposalBlock.Hash(), cs.ProposalBlockParts
} else {
// 다른 블록이면 nil 투표
```

```
cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)

}

}

} else {

// POL이 있는 제안인 경우 (이전 라운드에서 2/3 이상의 지지를 받은 블록)

if  cs.Proposal.POLRound < cs.LockedRound {

// POL 라운드가 잠긴 라운드보다 낮으면 nil 투표

cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)

} else {

// POL 라운드가 잠긴 라운드 이상이면 블록에 투표

cs.signAddVote(types.PrevoteType, cs.ProposalBlock.Hash(), cs.ProposalBlockParts

}

}

}
```

2.3 투표 수집 및 POL 형성

```
func (cs *State) addVote(vote *types.Vote, peerID p2p.ID) (added  bool, err  err

// 투표 추가

added, err = cs.Votes.AddVote(vote, peerID)

if !added || err != nil {

return
```

```
}

// Prevote 처리

if vote.Type == types.PrevoteType {

prevotes := cs.Votes.Prevotes(vote.Round)

// 2/3 이상의 특정 블록에 대한 투표가 있는지 확인

blockID, ok := prevotes.TwoThirdsMajority()

if ok {

// POL(Proof of Lock) 형성

if cs.isProposalComplete() || blockID.IsNil() {

cs.enterPrecommit(height, vote.Round)

}

} else if prevotes.HasTwoThirdsAny() {

// 2/3 이상의 투표가 있지만 특정 블록에 대한 것이 아닌 경우

cs.enterPrevoteWait(height, vote.Round)

}

}

}
```

3. 사전 커밋 단계 (Precommit Step)

3.1 단계 진입 조건

사전 커밋 단계는 다음 상황에서 시작됩니다:

- 특정 블록에 대해 2/3 이상의 사전 투표를 받았을 때 (POL 형성)

- 사전 투표 타임아웃이 발생했을 때

3.2 사전 커밋 로직

```
func (cs *State) enterPrecommit(height  int64, round  int32) {
// POL(Proof of Lock) 확인
blockID, ok := cs.Votes.Prevotes(round).TwoThirdsMajority()
if !ok {
// POL이 없으면 nil에 사전 커밋
cs.signAddVote(types.PrecommitType, nil, types.PartSetHeader{}, nil)
return
}
// Polka 이벤트 발행
cs.eventBus.PublishEventPolka(cs.RoundStateEvent())
if  blockID.IsNil() {
// nil에 대한 POL이면 nil에 사전 커밋
cs.signAddVote(types.PrecommitType, nil, types.PartSetHeader{}, nil)
return
}
// 특정 블록에 대한 POL인 경우
if  cs.LockedBlock != nil && cs.LockedBlock.HashesTo(blockID.Hash) {
// 이미 잠긴 블록과 같으면 재잠금
cs.LockedRound = round
cs.eventBus.PublishEventRelock(cs.RoundStateEvent())
```

```
cs.signAddVote(types.PrecommitType, blockID.Hash, blockID.PartSetHeader, cs.Lock
return
}
if  cs.ProposalBlock != nil && cs.ProposalBlock.HashesTo(blockID.Hash) {
// 제안된 블록과 같으면 블록 검증 후 잠금
if  err := cs.blockExec.ValidateBlock(cs.state, cs.ProposalBlock);  err != nil {
panic(fmt.Sprintf("Invalid block with POL: %v", err))
}
// 블록 잠금
cs.LockedRound = round
cs.LockedBlock = cs.ProposalBlock
cs.LockedBlockParts = cs.ProposalBlockParts
cs.eventBus.PublishEventLock(cs.RoundStateEvent())
cs.signAddVote(types.PrecommitType, blockID.Hash, blockID.PartSetHeader, cs.Prop
return
}
// 해당 블록을 가지고 있지 않으면 nil에 사전 커밋
cs.signAddVote(types.PrecommitType, nil, types.PartSetHeader{}, nil)
}
```

3.3 커밋 조건 확인

```
func (cs *State) addVote(vote *types.Vote, peerID p2p.ID) (added  bool, err  err
```

```
// ... (이전 코드)
// Precommit 처리
if vote.Type == types.PrecommitType {
precommits := cs.Votes.Precommits(vote.Round)
// 2/3 이상의 특정 블록에 대한 사전 커밋이 있는지 확인
blockID, ok := precommits.TwoThirdsMajority()
if ok {
if !blockID.IsNil() {
// 특정 블록에 대한 2/3 이상의 사전 커밋 → 커밋
cs.enterCommit(height, vote.Round)
} else {
// nil에 대한 2/3 이상의 사전 커밋 → 다음 라운드
cs.enterPrecommitWait(height, vote.Round)
}
} else if precommits.HasTwoThirdsAny() {
// 2/3 이상의 사전 커밋이 있지만 특정 블록에 대한 것이 아닌 경우
cs.enterPrecommitWait(height, vote.Round)
}
}
}
```

4. 커밋 단계 (Commit Step)

4.1 단계 진입 조건

커밋 단계는 특정 블록에 대해 2/3 이상의 사전 커밋을 받았을 때 시작됩니다.

4.2 블록 커밋 과정

```
func (cs *State) enterCommit(height  int64, commitRound  int32) {
// 커밋할 블록 확인
blockID, ok := cs.Votes.Precommits(commitRound).TwoThirdsMajority()
if !ok {
panic("enterCommit without +2/3 precommits")
}
if  blockID.IsNil() {
panic("enterCommit with nil block")
}
// 상태 업데이트
cs.updateRoundStep(commitRound, cstypes.RoundStepCommit)
cs.CommitRound = commitRound
cs.CommitTime = time.Now()
// 블록 실행 및 상태 적용
block := cs.ProposalBlock
if !block.HashesTo(blockID.Hash) {
// 블록을 다시 로드해야 하는 경우
block = cs.blockStore.LoadBlock(height)
}
```

```go
// 애플리케이션에 블록 적용
appHash, err := cs.blockExec.ApplyBlock(cs.state, types.BlockID{Hash: block.Hash
if  err != nil {
panic(fmt.Sprintf("Failed to apply block: %v", err))
}
// 상태 업데이트
cs.state.AppHash = appHash
cs.state.LastBlockHeight = height
cs.state.LastBlockID = blockID
cs.state.LastBlockTime = block.Time
// 다음 높이로 진행
cs.updateToState(cs.state)
}
```

5. 메시지 검증 로직

5.1 투표 메시지 검증

```go
func (cs *State) tryAddVote(vote *types.Vote, peerID p2p.ID) (bool, error) {
// 1. 기본 검증
if  err := vote.ValidateBasic();  err != nil {
return false, err
}
```

```
// 2. 높이 및 라운드 검증
if  vote.Height != cs.Height {
return false, ErrVoteHeightMismatch
}
// 3. 밸리데이터 검증
validator := cs.Validators.GetByIndex(vote.ValidatorIndex)
if  validator == nil {
return false, ErrInvalidValidatorIndex
}
// 4. 서명 검증
voteSignBytes := vote.SignBytes(cs.state.ChainID)
if !validator.PubKey.VerifySignature(voteSignBytes, vote.Signature) {
return false, ErrInvalidSignature
}
// 5. 중복 투표 검증
if  cs.Votes.GetVote(vote.ValidatorIndex, vote.Type, vote.Round) != nil {
// 중복 투표 감지 - 증거로 보고
cs.evpool.ReportConflictingVotes(existingVote, vote)
return false, ErrDuplicateVote
}
return  cs.addVote(vote, peerID)
}
```

5.2 블록 파트 검증

```
func (cs *State) addProposalBlockPart(msg *BlockPartMessage, peerID p2p.ID) (boo

// 1. 높이 및 라운드 검증

if  msg.Height != cs.Height || msg.Round != cs.Round {

return false, ErrInvalidBlockPart

}

// 2. 파트 인덱스 검증

if  cs.ProposalBlockParts == nil {

return false, ErrNoProposal

}

// 3. 파트 추가

added := cs.ProposalBlockParts.AddPart(msg.Part)

if !added {

return false, ErrInvalidBlockPart

}

// 4. 블록 완성 확인

if  cs.ProposalBlockParts.IsComplete() {

// 블록 재구성

blockBytes := cs.ProposalBlockParts.GetReader()

block := &types.Block{}

if  err := block.FromProto(blockBytes);  err != nil {

return false, err
```

```
}
cs.ProposalBlock = block
// 다음 단계로 진행
if  cs.Step <= cstypes.RoundStepPropose {
cs.enterPrevote(cs.Height, cs.Round)
}
}
return true, nil
}
```

6. 타임아웃 처리

6.1 타임아웃 종류

```
const (
timeoutPropose = "propose"
timeoutPrevote = "prevote"
timeoutPrecommit = "precommit"
timeoutCommit = "commit"
)
```

6.2 타임아웃 처리 로직

```go
func (cs *State) handleTimeout(ti timeoutInfo, rs cstypes.RoundState) {
// 타임아웃이 현재 상태와 일치하는지 확인
if ti.Height != rs.Height || ti.Round < rs.Round || (ti.Round == rs.Round && ti
return // 오래된 타임아웃 무시
}
switch ti.Step {
case cstypes.RoundStepNewRound:
cs.enterPropose(ti.Height, ti.Round)
case cstypes.RoundStepPropose:
cs.enterPrevote(ti.Height, ti.Round)
case cstypes.RoundStepPrevoteWait:
cs.enterPrecommit(ti.Height, ti.Round)
case cstypes.RoundStepPrecommitWait:
if ti.Round < cs.Round {
// 이전 라운드 타임아웃
cs.enterPrecommit(ti.Height, ti.Round)
} else {
// 현재 라운드 타임아웃 - 다음 라운드로
cs.enterNewRound(ti.Height, ti.Round+1)
}
}
```

```
}
```

7. 성능 최적화 포인트

7.1 조기 진행 (Early Advancement)

- 2/3 이상의 투표를 받으면 타임아웃을 기다리지 않고 즉시 다음 단계로 진행
- 블록 파트를 병렬로 전송하여 네트워크 지연 최소화

7.2 투표 집계 최적화

```
// 비트 배열을 사용한 빠른 투표 집계
type VoteSet struct {
votes []*types.Vote
votesBitArray *bits.BitArray
sum  int64  // 총 투표권
maj23 *types.BlockID // 2/3 이상 지지받은 블록 ID
}

func (voteSet *VoteSet) TwoThirdsMajority() (types.BlockID, bool) {
if  voteSet.maj23 != nil {
return *voteSet.maj23, true
}
return types.BlockID{}, false
```

```
}
```

7.3 메시지 전파 최적화

- 가십 프로토콜을 통한 효율적인 메시지 전파
- 중복 메시지 필터링으로 네트워크 대역폭 절약
- 우선순위 기반 메시지 큐잉

참고 자료

- Tendermint 합의 알고리즘 논문
- CometBFT 합의 구현 코드
- 비잔틴 합의 알고리즘 개요

이전 문서: 01-Tendermint-알고리즘.md

다음 문서: 03-비잔틴-장애-처리.md